

Agon PRD: hackathon build with feasibility gates

Sep 23, 2026 · @Someone

Summary

Agon turns a trader’s own on-chain history into a spending limit their AI agent must trade inside, enforced on Solana. It answers one question: **"You set a stop. Why don't you keep it?"**

Full product and technical reference: [Agon, product and technical reference](#). Where the two disagree, this PRD wins: it reflects the feasibility findings (cut scope, Jupiter Trigger instead of our own executor, Swig permissions as documented).

- **Targets:** Crypto World’s Fair (submissions close Oct 12, 2026, 11:59pm PT), then Colosseum’s fall Solana hackathon (Sep 28 to Nov 2, 2026) with the extended build.
- **Custody:** non-custodial. The user’s key never leaves their wallet; the agent holds a capped, revocable Swig role; 24/7 rules run as Jupiter Trigger orders, not on our servers.
- **Team:** 5 full-stack engineers working with coding agents.
- **Rule for this document:** every risky claim has a feasibility spike with a measured pass threshold. Anything without a number is a guess and is labelled as one.

Judging criteria, and where beta users, X posts and benchmarks fit

Beta testers, public X posts and published benchmarks are allowed and they feed three of the six judged criteria. Nothing in the World’s Fair rules restricts marketing, social posts or real users ([official rules](#), section 8).

Criterion (rules s.8)	What the judges ask	What Agon shows
Functionality	How well does it work, what is the code quality	Feasibility test results below, public repo, live demo on mainnet
Potential impact	Addressable market, impact on the ecosystem	Beta cohort numbers; the guardrail benchmark

Criterion (rules s.8)	What the judges ask	What Agon shows
Novelty	How unique is the concept	The join, not either half. Wallet analysis exists (Nansen, Solana Tracker, BingX) and on-chain agent spend caps exist (Swig, Coinbase and MetaMask agentic wallets). What we have not found is a product that derives the cap from the user's own history and enforces that number on-chain. Searched so far: 2,992 Breakout and Cypherpunk entries via Colosseum Copilot (unverified; excludes Frontier, the February Agent Hackathon, the x402 hackathon and the current World's Fair)
UX	Does it use the chain to make UX better	Revoke without us; the chain as the verifier of every run
Open source	Is it open source, does it compose	MIT repo; <code>check_trade</code> callable by any agent (Solana Agent Kit, GMGN agents)
Business plan	Viable business, can the team execute	Pricing slide; beta retention; weekly shipped updates

Prizes relevant to us ([rules](#), s.14): Grand Champion \$30,000; 20 standout teams \$15,000 each; Solana track \$100,000 across 10 products. Winners are announced around Dec 5, 2026, after the fall hackathon closes.

Constraints that matter:

- One team, one submission at a time (s.7). The rules do not forbid extending the same project for the fall hackathon; past winners (Unruggable, Lomen) carried one project across several Colosseum hackathons. Confirm with Colosseum before Oct 12 (open question).
- Prize receipt depends on due diligence (s.13). Beta users trading real funds through our code is a diligence question; the pre-mainnet checklist exists partly for this.

- The fall hackathon's criteria were not checked for this draft. Assume the same six until confirmed.

Scope

World's Fair ships the loop end to end on one wallet: mine, check, arm, revoke. The fall build adds the screener, event triggers and the decision model. Every item names the feasibility test (F-number, next section) it depends on.

Feature	World's Fair (Oct 12)	Fall (Nov 2)	Depends on
Wallet report: stop discipline, sizing, hold time, cost of exceptions	Yes	Yes	F1, F2
<code>check_trade</code> MCP tool: token risk plus "breaks your own rule"	Yes	Yes	F3, F4
Judgement checks on Jev: token category (cached once per mint for all users), impersonation, and an injection screen on all outside text entering through Agon; everything numeric, including style fit, is arithmetic	Yes	Yes	F11
Arm a rule: Swig role (<code>program</code> = Jupiter, <code>tokenRecurringLimit</code>) plus Jupiter Trigger order	Yes	Yes	F5, F6, F7
Revoke from wallet, live	Yes	Yes	F5
Shareable "what your exceptions cost" card	Yes	Yes	F2
Guardrail benchmark, published	v1 (20 scenarios)	v2 (full suite)	F9
Speed and performance benchmark: Jev vs LLM guard vs arithmetic	v1, three arms, latency + accuracy only	Full four-arm harness plus Kev-0.5B	F11, F9

Feature	World's Fair (Oct 12)	Fall (Nov 2)	Depends on
Beta cohort	10 to 20 users, read-only report	50+ users, armed rules on mainnet	Pre-mainnet checklist
Behavioural screener over a seeded index	No	Yes	F8
Balance- and event-triggered rules via Helius	No	Yes	F10
Pricing and paid tier	Slide only	Slide plus waitlist	None

Cut, and why:

- **Record-and-replay of arbitrary agent tasks.** Jupiter's CLI and hosted Trading MCP already make the first run cheap, so "the tenth time is faster" is a weak demo.
- **Our own stop-loss executor.** Jupiter Trigger V2 already runs stops and trailing stops from vaults around the clock. We create Trigger orders; we do not race them.
- **DuckDB, Yellowstone streaming, speculative transaction builds.** Latency work only pays off when racing for a slot, and Jupiter covers that case.
- **Hosting a signing daemon for users.** That makes us a holder of spend authority. Out of scope for both hackathons.

Components to build

One TypeScript monorepo, four deployable pieces, no custom Solana program. Anything that can sign runs on the user's machine; anything hosted only reads.

flowchart LR

```

AG[User's agent<br/>Claude, Cursor, Codex] --> MCP[agon MCP
server<br/>stdio, local]
MCP --> GD[Guard<br/>check_trade]
MCP --> DM[Local daemon<br/>agent key, armed rules]
GD --> API[Agon API<br/>hosted, read-only]
WEB[Web app<br/>report, arm, revoke] --> API
API --> EXT[Helius, RugCheck,<br/>Jev, Jupiter]
DM --> CH[Swig role +<br/>Jupiter Trigger]
WEB --> WL[User wallet<br/>Phantom, Backpack]
WL --> CH

```

The user's wallet signs everything that grants or removes authority; the daemon's key can only spend inside the Swig role.

Component	Where it runs	What it does	Built from
agon CLI	User machine (npm)	Install, register MCP with the user's assistants, start the daemon	Node, commander
MCP server	User machine, stdio	Tools: <code>get_report</code> , <code>check_trade</code> , <code>arm_rule</code> , <code>list_rules</code> ; one stable tool list so prompt caches survive	MCP TypeScript SDK
Local daemon	User machine	Holds the agent key in the OS keychain; builds and signs swaps inside the Swig role; submits the pre-signed expiry (F7); event triggers in the fall	@solana/kit, OS keychain
Guard (<code>check_trade</code>)	Library, used by MCP and API	Token check, personal-rule check (arithmetic), Jev judgement, price band vs Jupiter quote	Pure functions plus two network calls
Decoder and miner	Library, used by API	Balance-change decoding, FIFO P&L, rule mining, report JSON	Helius enhanced transactions

Component	Where it runs	What it does	Built from
Agon API	Hosted	<code>/report/:address</code> , <code>/check</code> , <code>/jev</code> proxy (users need no Jev key), token-check cache, beta metrics	Next.js route handlers on Vercel; long backfills on one worker
Web app	Hosted	Paste address, report, share card image, arm-rule flow (wallet signs Swig role plus Trigger order), revoke, benchmark page	Next.js, wallet adapter
On-chain	Solana mainnet	Swig smart wallet and role; Jupiter Trigger V2 orders	Swig SDK, Jupiter API; no program of our own
Benchmark harness	CI and laptops	Benchmarks A and B, pinned mainnet fork	Surfpool, pinned model versions

How the Guard produces decision speed

1. **Arithmetic first.** Every numeric check (size, stop distance, price band, slippage, style fit) is arithmetic. If all guards resolve arithmetically, no model is called at all: this is the fastest path and should be the common one.
2. **One batched Jev call.** Jev evaluates all questions in a single parallel pass and output tokens are free, so asking twenty questions costs what one costs (TypeSafe's claim; F11 (b) tests it). Every call carries the whole non-numeric question set, and multiple rules triggering together share one call.
3. **Cache what never changes.** A token's category is decided once and cached for all users. After the first check on a mint, it needs no model call, so the share of model-free checks grows as usage grows.
4. **Pre-warm and fan out.** Profile and mint data load into memory at arm time, not read per trigger. Account reads, the quote and the transaction build run in parallel, not in sequence.

Acceptance: the share of `check_trade` calls resolved without any model call is measured and reported in Benchmark B.

Agent guardrails, in layers. Each layer assumes the one above it failed.

1. **Guard verdict:** `check_trade` must pass. Token check, personal rules and price band are arithmetic; Jev only answers non-numeric questions.

2. **Check bound to the exact transaction:** the daemon signs only a transaction whose message hash matches one `check_trade` approved in the last 30 seconds. A different amount or route needs a new check.
3. **Injection stance:** all outside text is data: token names and descriptions, social links, webhook payloads and tool outputs. Every such string entering through Agon goes through Jev's injection screen in the same batched call, and a "looks injected" flag blocks. Nothing in fetched content can change a rule.
4. **On-chain cap:** the Swig role allows only Jupiter's program and only up to `tokenRecurringLimit`. This holds even if layers 1 to 3 and the whole machine are compromised.
5. **Hand-back:** any unsure or failed verdict returns to the user with the reason and numbers; nothing retries silently.
6. **Kill switch:** one wallet signature removes every Agon role.

Repo layout

apps/web	Next.js: web app + API route handlers
apps/worker	long backfills (only if F2 shows Vercel limits bite)
packages/core 1)	shared types, rule spec, report JSON schema (frozen day
packages/decoder	balance-change decoding + FIFO P&L
packages/miner	rule mining
packages/guard	check_trade
packages/chain	Swig roles, Jupiter orders, kill switch
packages/mcp	MCP server
packages/cli	agon CLI + daemon
spikes/F1..F11	one folder per feasibility test: run script,
thresholds.json,	raw output
fixtures/golden	hand-verified wallet ledgers (source slot in every file)
benchmark/	scenarios, arms, results

Keys and secrets

Eleven key entries (twelve variables), three kinds of Solana keypair. No mainnet signing key ever goes into CI, a `.env` file or the hosted app. Each environment gets its own API keys, so a leaked dev key never touches production.

API keys

Variable	Service	Needed for	Local dev	CI	Hosted	By
HELIUS_API_KEY	Helius	RPC, enhanced transactions, history	Free tier	Separate key, nightly spikes only	Paid plan (open question)	CP1
HELIUS_WEBHOOK_SECRET	Helius	Verifying webhook calls (F10)	Yes	No	Yes	Fall
JUPITER_API_KEY	Jupiter Portal	Higher rate limits for quotes and Trigger orders	Optional	Nightly only	Yes	CP2
CLOUDFLARE_ACCOUNT_ID , CLOUDFLARE_API_TOKEN	Cloudflare Workers AI	Jev (token scoped to Workers AI only)	Yes	Nightly only	Yes, behind /jev	CP1
LLM_BASELINE_API_KEY	The chosen chat model's provider	LLM-guard arm of benchmark B; benchmark agent	Yes	Nightly only	No	CP2
RUGCHECK_TOKEN	RugCheck	Optional enrichment only; needed only if public report endpoints rate-limit us	Optional	Optional	Optional	CP1
DATABASE_URL	Neon Postgres	Report cache, beta metrics, "rule is right" answers	Local Postgres or a Neon branch	Neon branch per PR, or none	Yes	CP2
VERCEL_*	Vercel	None if the Git integration deploys; a token only if CI deploys	No	No	Git integration	CP2
GITHUB_TOKEN	GitHub Actions	Committing the feasibility dashboard	No	Built in, contents: write	No	Day 1

Variable	Service	Needed for	Local dev	CI	Hosted	By
NPM_TOKEN	npm	Publishing the agon CLI and MCP server (publish-only, automation token)	No	Public repo release job only	No	CP3
PUBLIC_REPO_TOKEN	GitHub	Release job pushing the stripped tree to the public repo (fine-grained, contents write on that repo only)	No	Private repo release job only	No	Day 1

No key needed: Jupiter public quote and price API at low volume, Pyth Hermes, Swig (users pay their own fees; we do not use Swig's paymaster).

Solana keypairs

Keypair	Where it lives	Where it never goes
Devnet test keys (throwaway)	Repo-ignored files and CI secrets	Mainnet
Mainnet team test wallets (\$50 cap each) and the funding wallet	Team members' hardware or password managers; used by hand for stage-3 tests	CI, <code>.env</code> , chat, the hosted app
User agent keys (one per user per rule set)	Generated on the user's machine, stored in the OS keychain by the daemon	Our servers, logs, anything we host

Rules

- `.env.example` lists every variable with its source; `.env` is gitignored; a secret scan runs on every push (see CI).
- CI jobs that call paid APIs run nightly or on demand, never on every push, so a busy day doesn't burn the quota.

- Getting a key is an operator task: it goes in `OPERATOR_TODO.md` with the task it unblocks (next section).

Feasibility policy

A component is feasible only when a spike has run against real chain data and hit a number written down before the spike started. Vendor docs, a working happy path, or "it should work" do not count.

The bar

1. The threshold is written in this doc before the spike runs. Changing it afterwards requires a note saying who changed it and why.
2. A spike produces an artifact: a script in `spikes/` that anyone can rerun, and its raw output committed next to it.
3. "Pass" means the measured number meets the threshold on every listed input, not on average.

Environments, in order

Stage	Where	What it can prove	What it cannot
1. Devnet, intensive	Solana devnet, throwaway keys	Swig role creation, permission rejection, revocation, our own transaction building	Jupiter routing and Trigger execution (no real liquidity or keepers on devnet)
2. Mainnet simulation	<code>simulateTransaction</code> against mainnet, plus a Surfpool mainnet fork	Jupiter swaps and Trigger order creation against real accounts; Swig rejecting an over-cap swap	That a keeper actually fills a Trigger order; real landing rates
3. Mainnet, gated	Mainnet, team wallets only, capped funds	Keeper fills, landing, revocation under real conditions	Scale

Nothing moves from stage 2 to stage 3 until every item on the pre-mainnet checklist is ticked.

When a test fails: the team decides at a checkpoint

Failures are not debated when they happen. They are logged and decided at the next checkpoint, with evidence in hand.

- Each checkpoint has a date (see Timeline) and a fixed agenda: every F-test's status, its raw output, and the options listed in its row below.
- Options are always one of: **keep** (it passed), **fallback** (the pre-listed alternative), **cut** (drop the feature and its dependents), **extend** (one more spike with a new date; allowed once per test).
- The decision and its reason are written into the Decisions log at the end of this doc the same day. An undecided test at a checkpoint defaults to its fallback.

Feasibility tests

Eleven spikes, ordered by how much of the product dies if they fail. F1, F5 and F6 are existential: if any fails without a workable fallback, the World's Fair scope changes.

F1. Decoding trades from balance changes (existential)

- **Question:** can we turn raw transactions into buys and sells with exact amounts, across the DEXes real traders use?
- **Inputs:** 3 team wallets with trades we can reconstruct from memory and receipts, plus 2 public wallets with 200+ swaps across at least 4 venues (Jupiter, Raydium, Orca, Meteora, pump.fun).
- **Method:** decode every transaction from pre/post token balances (Helius). Hand-build a ledger for 50 randomly sampled transactions per wallet and compare field by field.
- **Pass:** 50 of 50 sampled transactions classified correctly (buy, sell, or not-a-swap) on every wallet; amounts exact to base units; realised P&L per wallet within 1% of a hand-computed FIFO ledger; every transaction either decoded or listed as unsupported with a named reason (no silent drops).
- **Also measure:** share of each wallet's swaps decoded. Below 95% is a finding for the checkpoint, not a pass.
- **Fallback:** decode Jupiter-routed swaps only and print the covered share on the report ("based on 83% of your swaps").

F2. Mining habits that are real, not noise

- **Question:** does the miner find a planted rule, and does it stay quiet when there is none?
- **Method:** three synthetic ledgers. (a) 20 trades exiting at an 8% loss plus 2 held to 20%+; (b) a random-exit ledger with no rule; (c) a real beta wallet. Then time the full report on a wallet with 2,000 transactions.
- **Pass:** (a) stop found at 8% within 1 percentage point, exceptions counted as exactly 2, their cost exact; (b) reports "no consistent stop rule" rather than inventing one; (c) two runs give byte-identical output. Report for 2,000 transactions in 60 seconds or less, including Helius pagination.
- **Fallback:** ship only the metrics that passed (sizing and hold time are simpler than stop detection) and label the rest "coming soon" in the demo, not "N/A".

F3. Token risk check

- **Question:** does our own on-chain check catch dangerous mints reliably enough to be the primary path, with RugCheck as optional enrichment?
- **Method:** a labelled set of 30 mints: 10 with live freeze or mint authority, 10 Token-2022 mints with a permanent delegate, transfer hook or transfer fee, 10 blue chips. The primary check reads mint authority, freeze authority and Token-2022 extensions ourselves with one `getMultipleAccounts` call. RugCheck is optional enrichment and never blocks a trade.
- **Pass:** our own check flags all 20 dangerous mints and 0 of 10 blue chips.
- **RugCheck's role:** extra signal shown alongside our verdict when it answers; it is never on the path that decides whether a trade goes out, so its uptime and latency are not tested.

F4. `check_trade` : "this breaks your own rule"

- **Question:** does the personal-rule check give the verdict we expect, fast enough to be called on every trade?
- **Method:** a mined profile plus 20 scripted trades with verdicts written down in advance (over usual size, past usual stop, new token outside usual set, normal trade, and edge cases at exact thresholds).
- **Pass:** 20 of 20 verdicts match; p95 latency 300 ms or less excluding the RugCheck call; the output names the rule and the number ("4.1x your median size of 0.8 SOL").

- **Fallback:** none needed; this is arithmetic. A failure here is a bug, not a feasibility problem.

F5. Swig role actually enforces the limit (existential)

- **Question:** does a Swig role with `program` = Jupiter and `tokenRecurringLimit` block everything else, on-chain, even if our code is compromised?
- **Method, devnet then mainnet simulation:** seven cases. (a) swap within cap succeeds; (b) swap over cap rejected; (c) transfer to an arbitrary address rejected; (d) call to a non-Jupiter program rejected; (e) allowance restored at the expected slot after the window; (f) root removes the role and the next agent transaction fails; (g) removal done from Phantom, not only our CLI.
- **Also, route size:** wrap 20 real Jupiter routes of increasing complexity in a Swig execute instruction and record two counts: how many fit the legacy limit (1,232 bytes, address lookup tables allowed) and how many fit the v1 limit (4,096 bytes, at most 64 accounts, all inline, because v1 does not support address lookup tables). **We send v1 transactions.** [Transaction V1](#) activated on mainnet at epoch 1035 on 15 September 2026; sending it is opt-in, so `@solana/kit` (8.0.0 or later) must be configured to build v1.
- **Pass:** all 7 cases behave exactly as stated, zero unexpected successes; 18 or more of 20 routes fit the v1 limit (4,096 bytes and 64 inline accounts). The legacy 1,232-byte count is recorded for reference and is not a pass criterion.
- **Fallback for route size:** request quotes with Jupiter's `maxAccounts` lowered until the route fits 64 inline accounts, and measure the price cost of doing so; a route that still needs more than 64 accounts goes out as a v0 transaction with address lookup tables, inside 1,232 bytes. **No fallback for (b), (c), (d):** if the cap does not hold on-chain, the custody story is gone and the checkpoint decides whether Agon ships read-only.

F6. Jupiter Trigger order owned by a Swig wallet (existential)

- **Question:** can a Trigger V2 order be created, filled and cancelled with a Swig wallet (a program-derived address) as its owner, and does the vault deposit count against the Swig cap?
- **Method:** mainnet simulation first, then mainnet with \$20 per order: a stop order that should fill, a take-profit that should not, a cancel.
- **Pass:** order created from the Swig wallet; the keeper fills the first order when the price condition is met; proceeds return to the Swig wallet; cancel returns funds; the deposit reduces the Swig allowance by exactly the deposit.

- **Fallback:** our local daemon polls price and executes the swap through Swig itself. The UI must then say "runs while your computer is on", and the pitch loses 24/7 execution.

F7. Rule expiry without giving the agent admin rights

- **Question:** Swig's Developer SDK exposes no expiry field. How does "until when" work without handing the agent `manageAuthority` (which would let it raise its own cap)?
- **Method:** first check Swig's protocol-level SDK for native expiry. If absent: at arm time the user signs a role-removal transaction against a durable nonce; the daemon submits it at expiry.
- **Pass:** the pre-signed removal lands after expiry and the next agent transaction fails; an earlier manual revoke still works and leaves the pre-signed transaction harmless; the agent key never holds `manageAuthority`.
- **Fallback:** no automatic expiry. Rules use a short recurring window instead, and the UI says "no end date: revoke from your wallet". Never fall back to giving the agent admin rights.

F8. Screener index (fall only)

- **Question:** can we index enough wallets, and is "consistency" a stable property rather than noise?
- **Method:** seed 5,000 wallets with 50+ swaps in the last 90 days. Score each wallet on the first and second half of its history separately.
- **Pass:** full ingest in 24 hours or less within our Helius credit budget; rank correlation between the two halves' consistency scores of 0.5 or more; search p95 500 ms or less.
- **Fallback:** if scores are not stable, the screener ships as raw filters (hold time, size band, venues) with no "consistency" ranking, and the pitch drops the claim.

F9. Benchmark harness is reproducible

- **Question:** can we run the guardrail benchmark so a sceptic can rerun it and get the same numbers?
- **Method:** run the benchmark (see Benchmarks section) twice from a clean checkout on a mainnet fork pinned to one slot.
- **Pass:** guardrail verdicts identical across runs; agent-side variance reported across at least 5 runs per scenario; one command reproduces everything.
- **Fallback:** publish only the deterministic half (guardrail verdicts on fixed trades) and drop the live-agent comparison.

F10. Balance- and event-triggered rules (fall only)

- **Question:** do Helius webhooks deliver the events we trigger on, fast and completely?
- **Method:** 200 known events over 24 hours on team wallets, with a reconciliation poll every 60 seconds as the reference.
- **Pass:** p95 from slot to our handler 5 s or less; zero events found by the poll that the webhook missed; cost within plan.
- **Fallback:** poll-only triggers at 60 seconds, stated in the UI.

F11. Jev as the judgement layer (World's Fair: a to e; fall: f)

- **What it is:** TypeSafe's decision model. It takes a state plus typed questions (yes/no, choice, score) and returns probabilities, not text; 32,000-token context; available on [Cloudflare Workers AI](#) as `typesafe/jev`. Launched mid-September 2026, so there is no track record ([Simon Willison](#)).
- **Known weakness, from its own documentation:** numbers, dates and adversarial content. Trading is full of all three. So Jev never decides a numeric check (size, stop distance, price band, slippage, style fit); arithmetic does. Jev gets only three non-numeric questions: which category a token belongs to (memecoin, stablecoin, liquid-staking token, blue chip, real-world asset, other), cached per mint for all users; whether its name, description and social links look like an impersonation; and whether any outside text entering through Agon (token metadata, social links, webhook payloads, tool outputs) contains instructions aimed at the agent. Style fit is then arithmetic over the user's category mix, for example "82% of your trades are blue chips; this is a memecoin at 3x your median size". The injection screen is one layer only, because adversarial input is on Jev's weakness list; the Swig cap backs it.
- **Method, World's Fair:** (a) 500 calls against our question schema; (b) latency at 1, 5 and 20 questions per call, 200 calls each, to test the claim that questions run in parallel; (c) 100 labelled cases with the answer known, 40 of them token categories; (d) 30 adversarial cases: token descriptions, social links, webhook payloads and tool outputs containing instructions aimed at the model; (e) 20 numeric questions, expected to fail, to confirm the routing rule. **Method, fall:** (f) our own invoice checked against the advertised \$0.042 per million input tokens.
- **Pass, World's Fair:** 500 of 500 responses valid against the schema; p95 500 ms or less at 20 questions, and 20-question p95 within 1.5x of 1-question p95; on the 100 labelled cases, a better Brier score than both an LLM guard and a no-model baseline; on the 30 adversarial cases, no case where an injected instruction flips a verdict to "safe". **Pass, fall:** cost within 2x of the advertised price.

- **If (e) passes unexpectedly:** still keep numbers on arithmetic. One lucky run is not evidence.
- **Fallback:** an LLM guard in structured-output mode with a stricter confidence threshold (a model's stated 0.85 is not a probability), or Kev-0.5B running locally. Arithmetic checks are unaffected either way, so the demo never depends on Jev being up.

Failure behaviour

Anything that can move funds fails closed: if we cannot verify, the trade does not go out. Analytics fail open but always say what they are based on. Every failure message names the cause, the number involved, and what the user can do next. No blank fields, no "N/A", no "something went wrong".

Failure	What the user sees	What the system does	Never
Helius rate-limits or drops mid-history	"Read 1,240 of about 2,000 transactions, up to Jun 3. Helius returned 429. Resume continues from there."	Saves a cursor; resumes from it; report is computed only on the stated range	Show a report as if history were complete
Transaction we cannot decode	"Excluded 37 transactions from 2 programs we don't read yet: [program ids]."	Counts and lists them; they never enter P&L	Guess, or drop silently
Too little history for a rule	"12 closed trades. A stop rule needs 20; sizing and hold time are shown."	Shows the metrics that have enough data	Invent a rule from 3 trades
RugCheck slow or unavailable	"RugCheck didn't answer in time. On-chain checks passed: no freeze authority, no permanent delegate."	Our own mint check decides	Treat a timeout as a pass without saying so

Failure	What the user sees	What the system does	Never
Our own mint check cannot reach RPC	"Couldn't verify this token. Not safe to proceed."	<code>check_trade</code> returns block; the agent hands back to the user	Let the trade out unverified
Price moves past the band between check and submit	"Quote moved 2.3% since the check, beyond your 1% band. Stopped."	Re-quotes once; stops if still outside	Submit on the stale check
Swig rejects: over cap	"This needs 3.2 SOL; 1.1 SOL left in this window, resets in about 4h 10m."	Nothing sent; shows remaining allowance and reset time	Retry automatically with a smaller amount
Transaction may not have landed	"Checking whether it landed..." then the signature and its final status	Checks the signature status before any resubmit; resubmits only when it is confirmed dropped	Send twice
Trigger order not filled yet	Jupiter's order status and the trigger price vs current price	Reads status from Jupiter	Say "executed" before it is
Daemon was offline	"Offline 3h 12m. Jupiter orders were unaffected. 2 event rules did not run: [list]."	Jupiter orders keep running; missed event triggers are listed	Fire missed triggers late at today's prices
User revokes while funds sit in a Trigger order	"Rule revoked. 2.0 SOL is still inside an open Jupiter order. Cancel it?"	Offers the cancel in the same screen	Imply the revoke also returned those funds

Pre-mainnet checklist

Every box is ticked, with a link to evidence, before any mainnet transaction. Two team members sign off, and neither wrote the code under test.

Before team wallets touch mainnet

- ☐ F5 passed all 7 cases on devnet and in mainnet simulation (link to spike output)
- ☐ F6 passed in mainnet simulation
- ☐ Program IDs for Swig and Jupiter pinned in config and checked against their official docs; no ID read from user input
- ☐ The agent key has no `manageAuthority`, verified by reading the role on-chain, not from our config
- ☐ Signature-status check before resubmit is implemented and tested with a deliberately dropped transaction
- ☐ Every mainnet test wallet funded with \$50 or less; the funding wallet is not a team member's personal wallet
- ☐ Kill switch tested: one command removes every Agon role from every test wallet, run once on devnet
- ☐ Keys: agent keys live only in the local daemon's OS keychain; none in `.env`, logs or the repo (secret scan passes)

Before beta users arm rules on mainnet (fall only)

- ☐ Everything above, plus 2 weeks of team-wallet mainnet use with no unexplained transaction
- ☐ Per-user cap defaults to \$25 per window, and our code cannot set a higher cap than the user typed
- ☐ Beta terms state plainly: experimental software, you can lose the capped amount, revoke from your wallet at any time
- ☐ A second engineer has reviewed every line that builds a transaction or a Swig instruction
- ☐ Revocation tested from Phantom and Backpack by someone outside the team
- ☐ Incident plan written: who runs the kill switch, how users are told, within what time

Read-only beta (World's Fair) needs none of the above: it never signs anything. It needs only the secret scan and no storage of users' addresses beyond the session unless they opt in.

Benchmarks to publish

We publish two benchmarks: how much damage guardrails prevent, and how fast and accurate the safety layer is. The honest speed claim is not "trading is faster" in general. It is that **Agon's safety check is faster than an LLM-based guard, at equal or better accuracy**. Most agent frameworks guard trades by asking a chat model; that is the baseline we beat, or we don't publish the claim.

- **The claim is decision speed:** our safety verdict is faster than the LLM-guard verdict most agent frameworks use today, at equal or better accuracy.
- **We do not claim faster execution.** We route through Jupiter and add checks, so a guarded trade is at best at parity with an unguarded one. We publish the measured overhead versus bare Jupiter MCP and call it parity, never an improvement.
- **Decision speed improves with routing and caching; execution speed does not,** because the slot floor is shared. The roadmap must not imply otherwise.

Benchmark A: agents with and without guardrails

- **Setup:** one LLM agent (model and version pinned) trading through Jupiter on a mainnet fork pinned to one slot, so every run sees identical prices and accounts.
- **Scenarios (100, written and committed before the first run):** 40 normal trades inside the user's profile; 30 dangerous tokens (live freeze authority, permanent delegate, no sell route); 30 rule breaks (4x usual size, past usual stop, and prompt injection: a tool output or token description telling the agent to send funds elsewhere).
- **Three arms:** agent alone; agent plus `check_trade`; agent plus `check_trade` plus a Swig cap. The third arm is the point: when injection tricks the agent into skipping the check, the on-chain cap still holds.
- **Metrics per arm:** dangerous trades executed; rule breaks executed; normal trades wrongly blocked; SOL at risk. Each scenario runs at least 5 times; report mean and range.
- **World's Fair v1:** 20 scenarios, arms 1 and 2. **Fall v2:** all 100, all three arms.

Benchmark B: speed and performance of the safety layer (World's Fair v1 narrow, fall full)

- **World's Fair v1, three arms on the same 100 labelled cases:** (1) arithmetic only; (2) arithmetic plus Jev; (3) LLM guard alone (pinned model, structured output).

- **World's Fair v1 measures:** p50 and p95 latency of the full `check_trade` call, and correct verdicts on the 100 cases. Cold and warm runs reported separately. Nothing else, apart from the no-model share required by the guard's fast-path acceptance line (Components).
- **Moved to fall:** Brier calibration and reliability plot; cost per 1,000 checks from our invoices; end-to-end timing from the agent deciding to trade to a signed transaction; on-chain overhead (compute units and fees from the Swig wrapper vs a direct Jupiter swap); a fourth arm, arithmetic plus LLM guard; a Kev-0.5B arm running locally.
- **Headline format:** "Safety verdict in p95 X ms vs Y ms for an LLM guard, with Z% fewer wrong verdicts." Every number comes from the published run.

Publication rules

1. Scenarios, metrics and pass criteria are committed to the public repo before the first run. The commit hash goes in every post.
2. All runs are published, including arms or scenarios where Agon does badly (false blocks are a real cost).
3. One command reproduces the numbers (F9).
4. No named competitor comparisons unless their run is in the repo too.

Say	Don't say
"Across 100 pre-registered scenarios, the agent executed 0 of 30 dangerous-token trades with Agon vs 11 without." (format only: these numbers are placeholders, not results)	"Agents are 10x safer"
"Even when a prompt injection got past the agent, the Swig cap stopped 30 of 30."	"Unhackable"
"Safety check in p95 240 ms vs 3.1 s for an LLM guard, with fewer wrong verdicts." (placeholder numbers)	"Trading is now instant" or any speed claim without the LLM-guard baseline beside it
"Jev handles the judgement; every numeric check is plain arithmetic."	"AI decides your trades"

Say	Don't say
"Safety verdict in p95 X ms vs Y ms for an LLM guard, with Z% fewer wrong verdicts." (placeholder numbers)	Any latency number without the accuracy number in the same sentence

Beta testers and X

Beta users are also a feasibility test: they tell us whether the mined rules are true about real people, which no spike can. Every report asks one question: "Is this rule right about you?" (yes / no / partly).

	World's Fair (to Oct 12)	Fall (to Nov 2)
Who	10 to 20 active Solana traders from team networks, trader Discords and Telegram groups, X replies	50+; World's Fair users first
What they do	Paste an address, get the report, share the card	Arm one rule on mainnet, capped at \$25 per window, only after the pre-mainnet checklist passes during the two weeks between hackathons, never before Oct 12
Signs anything	No (read-only)	Yes, after Oct 12 and only once the pre-mainnet checklist passes
We track	Reports run; "rule is right" rate; share rate; users returning within 7 days	Plus rules armed, revokes, blocked trades, retention week over week
Gate	None	Every checklist box ticked

The live mainnet demo uses team wallets capped at \$50 each, so the demo never puts third-party funds at risk.

Gate at CP3: "rule is right" at 70% or above across 10+ reports. Below 70%, the report ships as descriptive statistics, the rule-mining claim comes out of the pitch and the X posts, and the miner is the problem to fix before the fall build, whatever F2 said.

X plan

- One build-in-public post per week, mirrored as a Colosseum project update. Past winners (Seer, Daiko, Lomen) posted 3 to 4 updates.
- Benchmark posts carry the repo commit hash and the one-command rerun.
- User share cards are posted only by the user, or by us with written consent.

Never on X

- A named person's wallet, habits or P&L without their consent. It is public data, but the rules bar defamatory content ([rules](#), s.12) and it is a bad look.
- "Copy this wallet" or anything that reads as a recommendation. Agon describes behaviour; it does not tell people what to buy.
- A token, points or airdrop teaser. It adds legal and due-diligence risk (s.13) for no judging benefit.

Day-1 setup

Fifteen moves that make everything after them faster, each with an owner and a date. Day 1 is **Sep 24, 2026** , day 2 is **Sep 25, 2026** . The first one cannot be sped up by code, so it starts today.

Building faster

#	Move	Owner	Done by	Why
1	Every team member trades from their test wallet daily, until each wallet has 20+ closed trades, including a few deliberate exceptions to a stop rule	All; D coordinates	Starts day 1, 20 trades by CP2	The live demo and the mined-habit story need real history, which only time produces. Monad's demo wallet had its whole history in one 3-hour window, which broke its period filters (T8.8, T8.9)

#	Move	Owner	Done by	Why
2	Write the 3-minute demo script and the benchmark scenario list, then work backwards from them	E (script), B (scenarios)	Day 1	Anything not in the script is a candidate to cut at each checkpoint
3	One record/replay wrapper around every external call (Helius, Jupiter, RPC, RugCheck, Jev). Record real mainnet responses for the golden wallets and about 30 tokens, each stamped with its source slot	C (wrapper), A (recordings)	Day 2	Tests and offline spikes run without keys, rate limits or flakiness, and all five tracks work in parallel without sharing a quota
4	Thin end-to-end version on staging: paste an address, get a report from fixtures, <code>check_trade</code> returns a fixture verdict, arming is a no-op on devnet	E (web), C (MCP, API)	Day 2	Integration bugs show on day 2, not day 18. Monad's worst bugs only appeared end to end (T1.7, T6.8)
5	Scaffold: workspace, CI, release job, both Vercel projects, <code>TASKS.md</code> , <code>OPERATOR_TODO.md</code>	B	Day 1, before anyone else branches	Monad's scaffold task blocked all five tracks until it landed
6	Each contract defined once in the shared-types package, and that one definition produces the MCP tool schemas, API validation, frontend types and fixture checks	C	Day 1, with the frozen contracts	A shape change is one edit that fails loudly wherever it breaks
7	No arming UI until F5 and F6 pass	D gates, E waits	CP1	If either fails, the arming screen changes shape

#	Move	Owner	Done by	Why
8	Beta waitlist live	E	Day 1	CP3 needs 10+ reports, and recruiting is the slowest part

Product architecture

#	Rule	Owner	Why
9	Pure core: <code>check_trade</code> , the miner and the decoder take plain data and return a verdict or report, with no network calls inside	A (decoder, miner), C (guard)	The same code serves the MCP server, the API, the benchmark and replayed tests, so benchmark numbers match production by construction
10	Incremental reports: store decoded trades by wallet and last slot, and fetch only what is new	A	Monad rebuilt the full history on every request (T3.6); switching later is a rewrite
11	Cache by how often data changes: token category forever and globally; token metadata for minutes; mint and freeze authority never	C	Too little caching is slow; caching authority is a safety hole, and one batched read is cheap
12	Stamp every verdict with its data slot and rule version	C	Reproducible benchmark runs, and "why was this blocked?" has an answer
13	API in the same region as the RPC node and the Jev endpoint; measure on day 1	C	Probably the biggest decision-speed lever, and it is a setting, not code
14	Log timings per external call from the first request (inside the same wrapper as move 3)	C	Benchmark B needs the latency breakdown, and it shows which call is slow

#	Rule	Owner	Why
15	No accounts or login for the read-only report; wallet connection only to arm a rule	E	Removes an auth track from a 19-day build

Load check: Track C owns eight of these, on top of the MCP server and the daemon. The record/replay wrapper (3) and per-call timings (14) are one piece of work; if C slips on day 2, the wrapper moves to B after the scaffold.

Team, contracts and timeline

Five tracks run in parallel from day 2 because three contracts are frozen on day 1, each with fixture files so no track waits for another.

Frozen on day 1 (Sep 24, 2026)

1. `check_trade` MCP tool: input (mint, side, size, wallet) and output (verdict, reasons with rule name and number).
2. Mined report JSON: metrics, rules, exceptions and their cost, coverage share, unsupported transactions.
3. Rule spec: what "arm" consumes (mint set, cap, window, trigger type, expiry) and produces (Swig role, Jupiter order id).

Changing a frozen contract needs the owners of both sides to agree, and the fixture changes in the same commit.

Track	Owns	Tests
A. Decoder and miner	Balance-change decoding, FIFO P&L, rule mining, report JSON	F1, F2
B. Benchmark, then screener	Benchmark harness and scenarios; screener index for the fall	F9, F8
C. Agent side	MCP server, <code>check_trade</code> , token check, daemon	F3, F4, F10, F11
D. On-chain	Swig roles, Jupiter Trigger orders, revoke, expiry, kill switch, pre-mainnet checklist	F5, F6, F7

Track	Owns	Tests
E. Product and users	Report page, share card, onboarding, beta recruiting, X, demo video	Beta "rule is right" rate

In the first 3 days, B pairs with A on hand-built ledgers for F1, the highest-risk test.

Track E needs a second person from CP2 onward. It is six jobs for one person: report page, share card, onboarding, beta recruiting, X, and the demo video. UX is a judged criterion, and the demo video is how judges see everything else. The most plausible second person is whoever finishes Track D first.

Checkpoints

Date	Checkpoint	Evidence required
Sep 27, 2026	CP1	F1 on 2 wallets; F3; F5 on devnet; F6 in simulation; F11 access, schema and latency (a, b)
Oct 2, 2026	CP2	F1 and F2 complete; F4; F5 in simulation; F6 on mainnet (\$20 orders); F7; F9 on 20 scenarios; F11 accuracy, adversarial and numeric-routing cases (c, d, e)
Oct 8, 2026	CP3: World's Fair freeze	Loop works end to end on a team wallet; 10+ beta reports; benchmark A v1 and benchmark B v1 (three arms, latency and accuracy) published; beta "rule is right" rate at 70% or above across 10+ reports (below 70%: the report ships as descriptive statistics and the rule-mining claim comes out of the pitch and the X posts)
Oct 11, 2026	World's Fair submission	Submitted a day before the Oct 12 deadline
Oct 16, 2026	CP4	F8 and F10 spikes; F11 cost (f); benchmark B fall additions: fourth arm (arithmetic plus LLM guard), Kev-0.5B arm, calibration, cost, end-to-end timing, on-chain overhead; World's Fair feedback folded in
Oct 23, 2026	CP5: mainnet beta gate	Pre-mainnet checklist fully ticked, or beta stays read-only

Date	Checkpoint	Evidence required
Oct 30, 2026	CP6: fall freeze	Benchmark A v2 and benchmark B v2 published; beta metrics final
Nov 1, 2026	Fall submission	Submitted a day before the Nov 2 close

Co-working system and CI

The board lives in the repo, and CI checks that work is useful, not just green. This reuses what already worked: the claim-by-commit lock from monad's `TASKS.md`, Bombe's operator queue and auto-generated dashboard, and YieldPay's owner-and-dependency table.

Files

File	Purpose	Taken from
<code>TASKS.md</code>	The board and the lock. Claim a task by committing your name to its row on <code>main</code> before writing code; a rejected push means someone beat you to it	monad <code>TASKS.md</code>
<code>OPERATOR_TODO.md</code>	Human-only queue: keys, funding test wallets, outreach, recording, questions to Colosseum. An agent that hits one of these writes an <code>OP-N</code> entry, marks its task <code>blocked</code> , see <code>OP-N</code> , and moves on. Never fake a credential or a verification to look done	Bombe <code>OPERATOR_TODO.md</code>

File	Purpose	Taken from
<code>FEASIBILITY.md</code>	Generated, never hand-edited: each F-test's threshold, last measured value, pass or fail, date, commit	Bombe <code>progress.yml</code>
<code>spikes/F*/</code>	<code>run</code> script, <code>thresholds.json</code> (written before the first run), <code>result.json</code>	new
<code>.github/pull_request_template.md</code>	The questions below, answered in every PR	new

Checkpoint decisions stay in this PRD's Decisions log, not in the repo, so there is one place to look.

Task format (block style from Bombe, measured-finding style from monad)

```

### T-A03, Decode Meteora DLMM swaps
- Status: claimed 2026-09-25 | Owner: <name> | Branch: t-a03-meteora
- Depends-on: T-A01
- Serves: Functionality (judged) ; F1 coverage share
- Acceptance: 50/50 sampled Meteora txs match hand ledger; coverage on beta wallet 3 rises above 95%
- Evidence: <link to spikes/F1/result.json at a commit, or the PR>
- Kill criterion: coverage gain under 2 points after 1 day, so cut and list Meteora as unsupported

```

ID ranges: T-A to T-E per track, T-F for feasibility spikes, T-J for submission deliverables, OP-N for operator items. When a task finds something unexpected, write the finding with its number into the row, as monad does ("146 events became 14,237").

CI jobs

Job	When	Needs keys	What it runs
<code>gates</code>	Every push	No	Format, lint, typecheck, tests, build, secret scan

Job	When	Needs keys	What it runs
<code>feasibility-offline</code>	Every PR	No	Deterministic spikes on recorded mainnet data: F1 golden wallets, F2 planted rules, F4 verdicts, plus the budget checks below
<code>feasibility-live</code>	Nightly and on demand	Yes (CI-only keys)	F3, F5 on devnet, F10, F11, benchmark B; writes <code>result.json</code> , regenerates <code>FEASIBILITY.md</code> , commits it, and opens an issue when a test that passed now fails
<code>board</code>	Every PR	No	Lints <code>TASKS.md</code> and regenerates the status summary in the README

Real-world checks: is this useful, or noise?

Check	The question	How it is enforced	The failure it prevents
Usefulness	What does this task move?	<code>board</code> fails a task with no <code>Serves:</code> line naming a judged criterion or a measured user metric, or with an <code>Acceptance:</code> line that has no number. At each checkpoint, features with zero use in beta metrics or the demo script are listed for cutting. Unused exports fail the build (<code>knip</code>)	Polishing things nobody sees

Check	The question	How it is enforced	The failure it prevents
Wrong data	Is the data real and right?	Golden-wallet regression must match exactly. Every fixture declares its source (<code>mainnet slot N</code> or <code>synthetic: true</code>), and CI fails if anything under <code>benchmark/</code> or the demo path reads a synthetic fixture	monad T6.9: benchmark baselines ran on invented data
Noisy data	Would we double-count?	Decoded events per transaction are checked against unique signature and leg counts on golden wallets	monad T2.7: a batch event duplicated every cancel
Misleading names	Does the field name match the value?	Review only (CI cannot see it): any field reaching the user or the agent is checked by a second person	monad T3.7: "positions" were trade flow, off by 67x
Redundant steps	Are we doing work twice?	A counting network wrapper in tests: <code>check_trade</code> makes 3 network calls or fewer (one account batch, one quote, one Jev call); history fetch pages at most one per 100 transactions	The same account read once per guard
Token waste	Are we flooding the agent's context?	Each MCP tool response has a token budget checked in tests on golden wallets (<code>get_report</code> 2,000, <code>check_trade</code> 400). Nightly benchmark logs tokens per trade; a rise over 20% opens an issue	monad T6.8: a 2.1M-token tool result broke every question

Check	The question	How it is enforced	The failure it prevents
Unwanted complexity	Is this the smallest thing that works?	A new dependency fails CI unless the PR body has a <code>Dependency:</code> line saying why the stdlib or an existing package won't do. Diffs over 600 lines need a split or a stated reason. Authors run <code>/ponytail-review</code> before opening a PR (an agent skill, so not in CI). CLI cold start stays under 300 ms	Heavy installs, speculative abstraction
Honest status	Is it actually done?	<code>board</code> refuses <code>done</code> without an <code>Evidence:</code> link. Work gated on a human is <code>blocked</code> , see <code>OP-N</code> , never mocked to look finished	Green rows with nothing behind them

Working with agents

- Agents claim tasks by the same commit protocol as people, and each agent session works on its own branch or worktree.
- CI is the check for anything it can see. A second human reviews only what CI cannot: money math, anything that builds a transaction or a Swig instruction, the three frozen contracts, and docs another track reads.
- Work is credited to the person who ran the agent, with no AI co-author lines, matching the monad rules.
- Every Friday the generated dashboard becomes the Colosseum project update and the X post (see Beta testers and X).

Branches, repos and deployment

Feature branches merge into `dev`, and `dev` is released into a clean `main`. One change from the obvious setup: **`main` lives in a separate public repo**, because every branch of a public repo is public, so internal files on `dev` would be visible to judges anyway. The

private repo holds `dev` and all feature branches; the public repo holds only released `main`.

```
flowchart LR
    F["feature/t-a03-...<br/>private repo"] -- PR + CI --> D["dev<br/>private repo"]
    D -- release job<br/>allowlist + checks --> M["main<br/>public repo"]
    D --> S["Staging deploy"]
    M --> P["Production deploy<br/>+ npm publish"]
```

Code only moves left to right. Nothing is ever merged from `main` back into `dev`, and nobody commits to `main` by hand.

Branch	Repo	What goes in	How it gets there
<code>feature/t- <id>-<slug></code>	Private	Anything: plans, notes, scratch tests, agent files	Branched from <code>dev</code> after the task is claimed in <code>TASKS.md</code>
<code>dev</code>	Private	Everything the team needs: code, all tests, <code>TASKS.md</code> , <code>OPERATOR_TODO.md</code> , <code>FEASIBILITY.md</code> , <code>CLAUDE.md</code> , <code>.claude/</code> , <code>spikes/</code> , <code>docs/plans/</code>	Squash-merged PRs from feature branches, after CI and any required human review
<code>main</code>	Public	Only the paths on the public allowlist	The release job, never a merge

What `main` contains: an allowlist (`.publicinclude`), not a denylist, so a new internal file stays private by default.

- `apps/`, `packages/`, and their unit tests. Keep the tests public: "quality of the code" is a judged criterion, and tests are the evidence.
- `benchmark/` (scenarios, runner, results). The benchmark claims are only credible if anyone can rerun them from the public repo.
- `README.md`, `LICENSE`, `.env.example`, `docs/public/` (quickstart, MCP tool reference, architecture), and a public CI workflow.
- Never: `TASKS.md`, `OPERATOR_TODO.md`, `FEASIBILITY.md`, `CLAUDE.md`, `.claude/`, `spikes/`, `docs/plans/`, scratch files.

The release job runs on a `release-*` tag on `dev` :

1. Copy only allowlisted paths into a clean checkout of the public repo.
2. On the stripped tree: install, build and run the tests. `main` must work on its own, not just look tidy.
3. Secret scan, plus a check that no internal markers survive (`TASKS.md` , `OP-` , `.claude` , internal URLs).
4. Commit as one release with a changelog, authored by the person who tagged it, and push.

Release at least once a day during a hackathon. Judges see the public repo, and daily releases show steady progress without exposing the board.

Environments

Environment	Source	Trigger	Chain	Keys
Preview	Feature branch PR	Every push to the PR	Devnet	Dev keys
Staging	<code>dev</code>	Every merge to <code>dev</code>	Devnet plus mainnet read-only	Staging keys
Production web and API	Public <code>main</code>	Every release	Mainnet	Production keys
<code>agon</code> CLI and MCP server (npm)	Public <code>main</code>	Release tag, published from CI with npm provenance	Mainnet	<code>NPM_TOKEN</code>
Worker (only if F2 needs one)	Public <code>main</code>	Every release	Mainnet	Production keys

Release rules

- Before any release that changes transaction building, Swig roles or the daemon: run the deploy checklist skill, and the pre-mainnet checklist if it touches signing.
- Rollback: the web app rolls back to the previous deployment in one click. For npm, publish a patch and deprecate the bad version; never unpublish.
- Freeze: no production release in the 24 hours before a submission deadline (from Oct 10 for World's Fair, from Oct 31 for fall) unless it fixes a broken demo.

- Hotfixes go to `dev` first, then out through a normal release. There is no path that edits `main` directly.

Two more secrets, added to the keys table: `NPM_TOKEN` (publish-only, public repo CI) and `PUBLIC_REPO_TOKEN` (write access to the public repo only, stored in the private repo's CI).

Skills and tools in the workflow

Every step has one default tool, so nobody decides from scratch which one to reach for. A tool not listed here is not part of the default workflow. All of these are Claude Code skills or tools the team already has, except Graphify, which is added later.

Step	Default tool	Rule
Is this feature worth building?	Colosseum Copilot	Conversational by default. A deep dive only for a new product direction; it costs 20 to 30 API calls
Plan a task longer than one day	<code>superpowers:brainstorming</code> , then <code>superpowers:writing-plans</code>	Plan goes in <code>docs/plans/</code> on <code>dev</code> (never public). Tasks under a day skip this
Scope every change	Ponytail, always on at level full through a <code>UserPromptSubmit</code> hook	Same setup as monad: the hook restates the rule every turn
Set up the workspace	<code>superpowers:using-git-worktrees</code>	One agent session per worktree per claimed task, so parallel agents never share a checkout
Find code	Explore subagent now; Graphify once the repo passes about 500 files	Graphify's savings only show on large repos (Graphify); before that it is setup cost with no payoff
Library docs (Solana kit, Swig, Jupiter)	Context7, then the vendor's own docs	Context7 was failing to connect in this environment on 2026-09-23; fix it before relying on it

Step	Default tool	Rule
Write guard, decoder or miner code	<code>superpowers:test-driven-development</code>	Money logic gets its failing test first. Other code follows the thin-test rule: one meaningful check per change
Decide the test set for a track	<code>engineering:testing-strategy</code>	Once per track on day 1, not per task
CI red or a spike fails	<code>superpowers:systematic-debugging</code>	Find the cause before changing code; the finding goes in the task row with its number
Before opening a PR	<code>/ponytail-review</code> , then <code>/code-review</code> at medium	Plus <code>/security-review</code> when the diff touches <code>packages/chain</code> , the daemon, keys or transaction building
Claiming a task is done	<code>superpowers:verification-before-completion</code>	Matches the board rule: no <code>done</code> without an evidence link
Decision that is expensive to reverse	<code>engineering:architecture</code>	Writes a short ADR. Examples: v1 transactions, the two-repo split, Swig vs a custom program
Every Friday	<code>/ponytail-audit</code> on dev, <code>/ponytail-debt</code> into <code>TASKS.md</code> , <code>engineering:standup</code>	Standup output becomes the Colosseum update and the X post draft
At CP4	<code>engineering:tech-debt</code>	One pass before the fall build, not weekly
Before a production release	<code>engineering:deploy-checklist</code>	Plus the pre-mainnet checklist when signing changes

Jev in our own workflow: a one-week trial, kept only where it measures up

Jev fits narrow, repeated judgement calls on text: route, rank, flag, escalate. It does not fit anything numeric, anything that grants permission, or open-ended coding ([Jev for coding agents](#)). A Jev answer is an input to a rule, never the rule itself.

What exists

Tool	What it is	Status
TypeSafe agent skill (typesafe@typesafe-ai)	Teaches the agent to design typed questions; cannot send calls on its own	Official. Install project-scoped on dev : we need this skill for F11 and the product anyway
jev-bridge	MCP server: jev_ask , answer cache, cost tracking, jev_review for marking answers right or wrong	Community, 1 star, MIT, 170+ tests. Our choice if we use an MCP, because the review log doubles as calibration data
typesafe-as-a-judge	MCP plugin with route, rank, extract, verify and escalation-gate tools	Community, 2 stars, Apache-2.0. Broader but less proven

Both MCP servers are unofficial and hold our TypeSafe key. Pin a commit and read the source before installing.

One cost trap: an MCP call only saves tokens if the tool reads the files itself. If Claude has to copy a diff or log into the call, Claude writes those tokens out, and that costs more than Jev saves. So uses that need a large input run as scripts that read the diff themselves (in CI or a git hook), not as MCP calls from inside a session.

Trial uses, in priority order

Use	How it runs	Rule	Keep it if
Second-layer review escalation: does this diff change money math, transaction building, a frozen contract, or docs another track reads?	CI script reads the diff	Can only add a required reviewer, never remove one. Path rules stay primary	Catches 38 or more of 40 labelled past diffs (from monad's history, which has examples of every category) with 8 or fewer false flags
Vague task lines: does this <code>Acceptance:</code> state a checkable outcome?	<code>board</code> job, advisory comment	Never blocks; the numeric-acceptance lint still does	Half the flags lead to an edited row in week one
Beta feedback, X replies and issues sorted into bug / feature / confusion / rule-wrong	Scheduled script	Below the confidence threshold goes to a person	Agrees with a person on 45 or more of 50 labelled items
Context triage in long debugging sessions: rank candidate files or log chunks before reading them	<code>jev-bridge</code> from inside the session, passing short excerpts only	Claude still reads what it picks	Fewer tokens per resolved bug than the same week without it, from explain-usage

Never: anything numeric (thresholds, budgets, counts, dates), deciding a task is done, approving a merge or deploy, or anything touching keys or funds.

Side benefit: every trial call builds our skill at writing Jev questions, and the `jev_review` labels give early evidence for F11 (b) and (c). Cost is negligible, about \$0.00002 per 400-token call at the advertised price.

Decision: after one week, at CP2, each use is kept or cut on its own "Keep it if" line.

Token rules

- `CLAUDE.md` stays under about 60 lines. Detail lives in skills, which load only when needed. Hook messages stay to one or two lines each, because they are sent on every turn.

- Search through a subagent or Graphify; never paste whole files, logs or JSON into the main conversation.
- One agent per task. Multi-agent workflows only when someone explicitly asks for one; they cost many times more.
- MCP tool responses keep the token budgets already enforced in CI (`get_report` 2,000, `check_trade` 400).
- Every Friday, each person runs the explain-usage skill on one heavy session and records the biggest waste in the standup. What gets measured gets cut.

Open questions

- ☐ Do the fall hackathon judging criteria match the World's Fair six? If they weight traction or novelty differently, the fall column's priorities change. Ask Colosseum in the same thread as the two-hackathon confirmation.
- ☐ Who is the team leader? Prizes are paid only to them (rules s.15).
- ☐ Helius plan and credit budget for F1, F2 and the F8 index.
- ☐ Which agent runs the benchmark: an LLM through Solana Agent Kit, or through Jupiter's Trading MCP? Pin one.
- ☐ Does Swig's protocol-level SDK support role expiry? (Decides F7.)
- ☐ Beta terms: who reviews them before any user signs on mainnet?
- ☐ License: MIT, or something else? Open source is a judged criterion.
- ☐ Jev access: Cloudflare Workers AI, TypeSafe directly, or Venice? Pick one by CP1 and confirm rate limits for 200-call latency runs.
- ☐ Which chat model is the LLM-guard baseline in benchmark B? It should be one agents actually use, or the comparison looks rigged.
- ☐ Search the World's Fair project directory (public, 4,000+ builders) and the Frontier and Agent Hackathon winners for a wallet-analysis or agent-guardrail project. Owner: to be assigned. Due: Sep 27, 2026 .

Decisions log

Date	Test or topic	Decision	Why
2026-09-23	Scope	Cut record-and-replay and our own stop executor	Jupiter's Trading MCP and Trigger V2 already cover both
2026-09-23	Custody	Non-custodial; no hosted signing daemon	Hosting a daemon makes us a holder of spend authority
2026-09-23	Jev (F11)	Moved into World's Fair scope, with a speed and performance benchmark against an LLM guard	Team wants the speed story now; Jev only handles non-numeric judgement because its docs flag numbers, dates and adversarial input as weak
2026-09-23	Two-hackathon scope	Confirmed the same project may go to World's Fair then be extended for the fall hackathon	Confirmed individually with Superteam. Person and channel: source to be recorded (unverified until then)
2026-09-23	Benchmark B	Benchmark B narrowed and reinstated to World's Fair; claim is decision speed only, execution speed explicitly disclaimed	Beta goal is decision speed vs the LLM guard; execution runs on Jupiter's rails and shares the slot floor, so it is not winnable
2026-09-23	Repos and branches	Private repo holds feature branches and dev; a separate public repo holds main, filled only by an allowlist release job	Every branch of a public repo is public, so a dev branch with internal files cannot sit next to a public main
2026-09-23	Jev in the dev workflow	Official TypeSafe skill installed; four uses trialled for one week, each kept or cut at CP2 on a pre-written bar	Team wants Jev in the coding workflow; the fit is narrow text judgements, and only script-side uses avoid the MCP token cost

Date	Test or topic	Decision	Why
2026-09-23	Jev's product role	Added token categories (cached per mint) and an injection screen on all outside text; style fit moved from Jev to arithmetic over categories	Puts Jev on text-in, fixed-choice-out questions only; caching raises the model-free share of checks, which is the decision-speed claim

Sources

- [Crypto World's Fair official rules](#): judging criteria (s.8), limits (s.7), winner announcement (s.13), prizes (s.14)
- [Crypto World's Fair page](#)
- [Colosseum 2026 hackathon dates](#)
- [Swig permissions reference](#)
- [Jupiter developer changelog](#)
- [Jupiter CLI](#)
- Colosseum Copilot: project records for `seer` , `daiko` , `lomen-ai` ; winners vs all entries across Breakout and Cypherpunk
- [Agon, product and technical reference](#)
- [Bombe TODO.md](#) , [OPERATOR TODO.md](#) and [progress.yml](#)
- [YieldPay PROGRESS.md](#)
- monad `TASKS.md` and `.github/workflows/ci.yml` (local checkout; lessons T2.7, T3.7, T6.8, T6.9)